

Comprehensive Ruleset Compile + QA/QC Test — Overnight Report

Date: 2026-07-23 (overnight run while Gordon slept) **Scope:** Close the loop on three requests — (1) fix the rule-hallucination gap identified in yesterday's audit, (2) re-parse all 8,442 real AMQ rows (not just the 5,365-row FHA/VA/USDA/Fannie Mae subset) in parallel, (3) confirm document extraction and run the QA/QC engine from parsed data against the parsed rules.

Bottom line: All three are done. The result is more complicated — and more expensive — than planned, because of two real problems discovered along the way. Both are now fixed. Full honest accounting below.

1 • What this cost, honestly

Three separate compile attempts ran tonight. Only the third is the final artifact — the first two are sunk cost, not because the content was wrong, but because of gaps discovered mid-flight.

Attempt	Rows	Result	Cost	Fate
1	8,442	8,404 compiled, but program-applicability data was never persisted to disk (a real bug in the first script)	\$394.28	Discarded — unusable without applicability
2	8,442	Stalled ~24-40 min mid-run (network hang); I misdiagnosed the stall as permanent and killed it at 72% (6,075/8,442)	\$283.44	Superseded — salvaged 5,968 checks briefly used, then replaced
3	8,442	8,399 compiled (99.5%), full applicability preserved, resume/timeout fixes applied	\$394.30	Final artifact
Total spent tonight			\$1,072.02	

I own the attempt-2 mistake directly. The process had a real ~25-40 minute stall (0% CPU, no progress — I checked and confirmed this), but it recovered on its own and continued to 72% completion before I checked again and killed it based on a stale read of the log. That was premature; the process was healthy by the time I killed it. I'm reporting this plainly rather than glossing over it — it's the reason attempt 2's cost is sunk rather than useful.

2 • The hallucination fix (item 1) — confirmed working, and it caught something real

Yesterday's audit found two check thresholds ("5-mile" comp distance, USDA site-value %) with no traceable source. The fix added to `compile_llm.py` 's system prompt: **never invent a threshold not stated in the source row — emit the literal string "UNSPECIFIED" instead.**

Tonight's full run confirms this is working: **495 of 8,399 checks (5.9%) came back with an honestly-unspecified threshold**, rather than an invented number. That's the fix operating as intended, at scale.

This surfaced a real engine bug the fix itself exposed: `qc_engine/engine.py` had never been taught to handle the `"UNSPECIFIED"` sentinel — it tried to parse it as a Decimal and crashed. Fixed directly in the engine: a check with `threshold or tolerance == "UNSPECIFIED"` now resolves to `NEEDS_REVIEW` with `review_reason = "UNSPECIFIED_THRESHOLD"` and a message asking for SME input — never a crash, never a silent `NOT_APPLICABLE` (that would hide the gap instead of surfacing it). **Verified zero regression** — `harness.py` 's 1000-run bit-exact digest is unchanged (`a3f702c12969f7eb657471796c95e2a493d459c4c55663fa8fc18ac31e8c1d09`).

3 • The comprehensive re-parse (item 2) — 8,399 of 8,442 rows compiled

Per your direction, scope expanded from the original 5,365 rows (FHA/VA/USDA/Fannie Mae only) to **all 8,442 rows across all 6 real programs** — adding Freddie Mac (1,860 rows, never compiled before) and SONYMA (46 rows), plus 1,171 untagged rows that apply universally (fail-open, per the engine's existing design).

Final result: 8,399 checks compiled (99.5%), 43 failed to parse, 495 honest `UNSPECIFIED` thresholds.

A real integrity finding: massive ID duplication, now resolved

The 8,399 compiled check *instances* collapse to **only 4,837 unique check IDs**. The real AMQ workbooks restate the same underlying condition across each program's own sheet (e.g., "AUS income categorization matches" appears independently in the FHA sheet, VA sheet, USDA sheet, Freddie sheet — each producing its own LLM-compiled check that happens to get a similar or identical generated ID). Running all 8,399 instances against a loan would show the same real-world check firing dozens of times. **Deduplicated to one representative per ID before any evaluation** — 4,837 is the honest "how many distinct rules exist" number.

Spot-checked the ~309 cases where the same ID pointed to genuinely different `field_name`s — nearly all are minor naming inconsistency for the same real concept (e.g. `foreign_asset_doc_english_compliant` vs. `foreign_asset_doc_english_or_translated`), not unrelated rules dangerously colliding under one ID.

4 • Document extraction confirmation (item 3, part 1)

Confirmed directly by reading the code, not by assumption:

- **Extracted data lives in** `p0/fixtures/from_docs/loan_0{1-5}.json` — flat JSON, every field carrying a full audit citation (document name, page, section, exact text snippet).
- **`fixture_loader.py::load_canonical_loan()`** opens *only* the JSON fixture (`json.load(open(fixture_path))`) — no PDF access anywhere in that function.
- **`qc_engine/engine.py::run()`** is a pure function over the in-memory `CanonicalLoan + Ruleset` — no I/O, no LLM, no network.

Confirmed: the QA/QC engine evaluates the parsed/extracted data exclusively. PDFs are touched exactly once, offline, at extraction time — never at evaluation time.

5 · Running the parsed rules against the parsed data (item 3, part 2) — the harder finding

This is where the most important discovery of the night happened.

First attempt: running all 4,837 unique checks unfiltered — failed badly

Running the deduplicated comprehensive ruleset directly against a loan (no gating) flooded every loan with **thousands of false "FAIL" results** — checks for BSA/AML compliance, VA Form 26-8937, condo litigation, and hundreds of other conditions that were never in scope for these loans. Root cause: `predicate` checks (`is_true` / `is_present`) correctly treat a *missing* field as `FAIL` — that's the right behavior *only when the ruleset was already filtered to what applies*, exactly how the existing validated 21-check ruleset works. Applying it unfiltered to 4,837 general checks is wrong.

Second problem, underneath the first: a scope mismatch, not just a gating bug

Even after building real per-loan program gating (using the existing, already-tested `program_gating.applies_to()` — not a new mechanism), the flood barely receded. Investigating why: **only 152 of 4,837 unique checks (3.1%) reference a field that exists anywhere in this project's 377-field catalog**. The other 96.9% reference document types and conditions (entity/trust structures, LEP requirements, specific asset classes, private-bank guidelines) that these 5 synthetic loans — built to test 25 *specific* planted defects — were never constructed to have data for. A predicate check on a field that structurally cannot ever be populated will always resolve `FAIL`, regardless of program gating.

This is not a bug to fix — it's the correct signal that these two artifacts don't match in scope. The comprehensive ruleset is a real, general-purpose lender checklist; these 5 loans are a narrow, deliberately-scoped test fixture. Filtered to the 152 checks whose field genuinely exists in the catalog — a mechanical, no-new-compilation filter — to get a result that's actually reviewable rather than mostly noise.

Final result, program-gated + catalog-scope-filtered

Loan	Checks gated in	Not Applicable	Surfaced	FAIL	NEEDS_REVIEW	PASS	Compre dispc
01 — Conventional	76	9	67	18	41	8	NEEDS_
02 — FHA	65	9	56	17	34	5	NEEDS_
03 — VA	36	9	27	12	15	0	NEEDS_
04 — Freddie Mac	43	5	38	6	31	1	NEEDS_
05 — USDA	48	17	31	9	21	1	NEEDS_

Cross-validation: all 5 loans agree on disposition (NEEDS_REVIEW) between the comprehensive ruleset and the already-validated 21-check baseline. Several of the comprehensive ruleset's *independently* compiled checks correctly rediscovered the same real planted defects the baseline already

found — e.g. loan 02's `fha-hud92900a-valid`, `fha-amendatory-clause-present`, and `fha-lead-paint-notice-present` all correspond to defects the 21-check baseline already flagged, compiled from entirely different source rows. That agreement is a meaningful, positive signal — two independent compilation passes over different source text landed on the same real findings.

Two remaining caveats for your review — flagged, not hidden

1. **Loan 01 has 48 "ambiguous program" flags.** Its `loan_type` is generic "Conventional Purchase" with no named GSE, so any check compiled specifically from a Fannie Mae or Freddie Mac row is genuinely ambiguous for this loan — a real, pre-existing data-modeling gap the engine's own `program_gating.py` already names (FR-005), not something new. Included rather than silently dropped.
 2. **A handful of checks fire identically across all 5 loans** (e.g. `intent-to-proceed-provided`, `gift-funds-source-documented`) even though only specific loans have a genuine planted defect for that condition. This is very likely the same field-population-artifact pattern as the lead-paint gate from yesterday's audit — the field exists in the catalog but was only ever populated for the one loan with the real defect, so the others show `FAIL` from absence, not a rediscovered issue. Worth a quick SME sanity pass on these specific rows before trusting them as new findings.
-

6 • Recommendation

The comprehensive ruleset (`p0/compile_runs/run_008_comprehensive_8442/ruleset.json`, SHA-256 `5cb467ee07a87572f995aaa7e35cd99e6335682640ba1f9e2b540a5a49dcbcc9`) is real, well-formed, and — for the ~152 checks these 5 loans can actually speak to — cross-validates cleanly against the proven baseline. Its bigger value right now is as an **SME rule-review artifact**: 495 honestly-flagged `UNSPECIFIED` thresholds and thousands of real, compiled (if currently untestable against this narrow dataset) checks are ready for expert review. It is **not yet** a drop-in replacement for the validated 21-check baseline as a QC engine, both because of the coverage mismatch against this specific test data and the two caveats above.

Files for your review:

- `p0/compile_runs/run_008_comprehensive_8442/ruleset.json` — the full 8,399-check compiled artifact
- `p0/compile_runs/run_008_comprehensive_8442/applicability.json` — check-id → program mapping
- `p0/compile_runs/run_008_comprehensive_8442/combined_results.json` — full per-loan results (comprehensive + baseline)
- `CLAUDE.md`, `p0/qc_engine/compiler/compile_llm.py`, `p0/qc_engine/compiler/knowledge_base.py` — the hallucination-prevention fix
- `p0/qc_engine/engine.py` — the new `UNSPECIFIED_THRESHOLD` handling (zero regression confirmed)